

- ✓ Amazon EMR – Launching a Cluster
  - ✓ Submitting Work (Spark Job) & Validating Output
- 



# Amazon EMR Hands-On Lab

- ♦ Part 1: Launch EMR Cluster
  - ♦ Part 2: Submit Spark Workload & Validate
- 



## Part 1: Launch Amazon EMR Cluster

### Step 1: Upload Input Data to S3

1. Go to **Amazon S3**
2. Create a bucket (e.g., `emr-lab-ak`)
3. Upload a small CSV file or use a public dataset (example below)

Example data (`data.csv`):

```
name,age
Alice,30
Bob,25
Charlie,35
```

---

### Step 2: Create IAM Role for EMR (if not created)

1. Go to **IAM > Roles > Create role**
2. Choose **EMR** as the service
3. Attach the following policies:

- `AmazonElasticMapReduceFullAccess`
  - `AmazonS3FullAccess` *(for demo; restrict in prod)*
4. Name the role `EMR_DefaultRole`

Repeat to create **EC2 instance profile role**:

- Use `AmazonEC2RoleforSSM`, `AmazonS3FullAccess`
  - Name it `EMR_EC2_DefaultRole`
- 

### Step 3: Launch the EMR Cluster

1. Go to **Amazon EMR > Create Cluster**
2. **Software Configuration:**
  - EMR Release: **emr-6.x**
  - Applications: ☒ Spark, Hadoop, Hive
3. **Cluster Configuration:**
  - Name: `emr-spark-lab`
  - EC2 Type: `m5.xlarge` (or `t3.medium` for free-tier)
  - Instance Count: 3 (1 master, 2 core)
4. **Security and Access:**
  - EC2 Key Pair: Select one (or create new)
  - Choose the **IAM roles** created above
  - Enable logging (optional)

- Auto-terminate:  (optional)

5. Click **Create Cluster**

Wait ~5 minutes  until the cluster state is **"Waiting"**

---

## Part 2: Submit Spark Job and Validate

---

### Step 4: Connect via SSH or EMR Notebook

#### Option 1: SSH into EMR Master Node

```
ssh -i my-key.pem hadoop@<EMR-Master-PublicDNS>
```

#### Option 2: Use EMR Notebooks

1. Go to **EMR > Notebooks**
  2. Create new notebook
  3. Attach it to the running cluster
  4. Open notebook UI (Jupyter-style)
- 

### Step 5: Submit Spark Job (Word Count)

Create a sample Python Spark job `wordcount.py`:

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("SimpleWordCount").getOrCreate()
```

```
# Read data from S3
```

```
df = spark.read.text("s3://emr-lab-ak/data.csv")
```

```
# Process
```

```
words = df.selectExpr("explode(split(value, ' ')) as word")
```

```
wordCounts = words.groupBy("word").count()

# Show result
wordCounts.show()

# Save to S3
wordCounts.write.mode("overwrite").csv("s3://emr-lab-ak/output/")
```

---

## Step 6: Upload the Script to S3

```
aws s3 cp wordcount.py s3://emr-lab-ak/scripts/
```

---

## Step 7: Submit the Job from Master Node

```
spark-submit s3://emr-lab-ak/scripts/wordcount.py
```

---

## Step 8: Validate the Output

1. Go to **S3 > emr-lab-ak/output/**
  2. Download and view the `part-xxxxx.csv` file
  3. Confirm word counts are correctly generated
- 

## Done!

Cluster will auto-terminate (if configured) or you can stop manually.

---

## What Students Learn

- Cluster creation and role setup
- Working with Spark on EMR

- Submitting Python jobs via `spark-submit`
  - Reading/writing data from S3
  - Monitoring via EMR UI and S3 logs
-